1) Creación de la aplicación según tutorial

Crear aplicación de funciones (Consumo flexible) ...

Seleccione una cuenta de almacenamiento o cree una nueva. Las cuentas deben ser compatibles con el almacenamiento de blobs, colas y tablas. Más información

Cuenta de Storage * (Nuevo) scalabilitylab2890e

Crear nuevo

Puede configurar redes en nuevas cuentas de almacenamiento al habilitar la integración de red virtual y configurar redes de acceso saliente en la aplicación de funciones.

Configuración de diagnóstico

Configure las opciones de diagnóstico para habilitar la supervisión de datos para la cuenta de almacenamiento. Obtener más información

Configuración de diagnóstico de Blob service

- (●) **Configurar más adelante (recomendado para controles personalizados)** Seleccione esta opción para controlar los destinos de registro, las directivas de retención y los registros y métricas específicos.
- ( ) **Configurar ahora (recomendado para controles básicos)** Seleccione esta opción para configurar Azure Log Analytics con almacenamiento, registros de escritura y métricas de transacciones para Blob service. Todavía puede modificar esta configuración más adelante.

Datos básicos | Storage | Azure OpenAI | Redes | Supervisión | Durable Functions | Implementación | Autenticación | Etiquetas | **Revisar y crear**

Resumen

Aplicación de funciones
de Microsoft

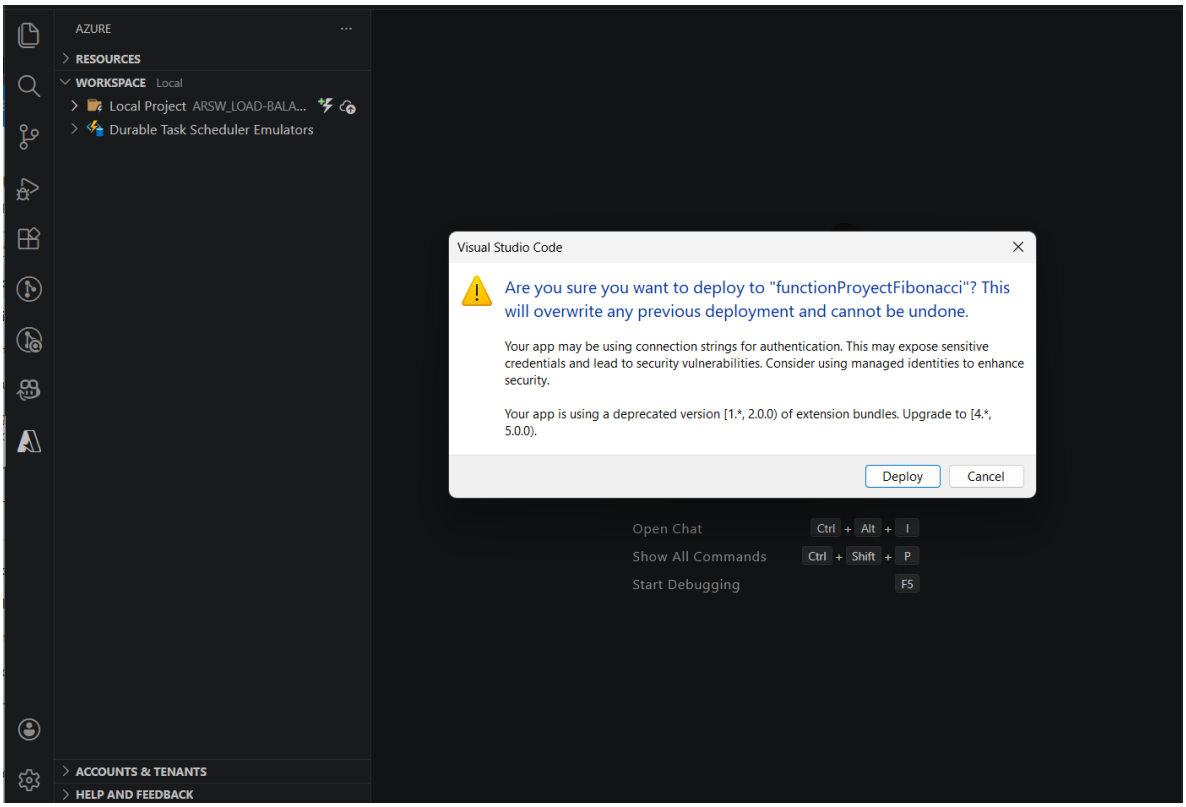
La autenticación básica para esta aplicación está deshabilitada actualmente y puede afectar a las implementaciones. Haga clic aquí para más información.

Detalles

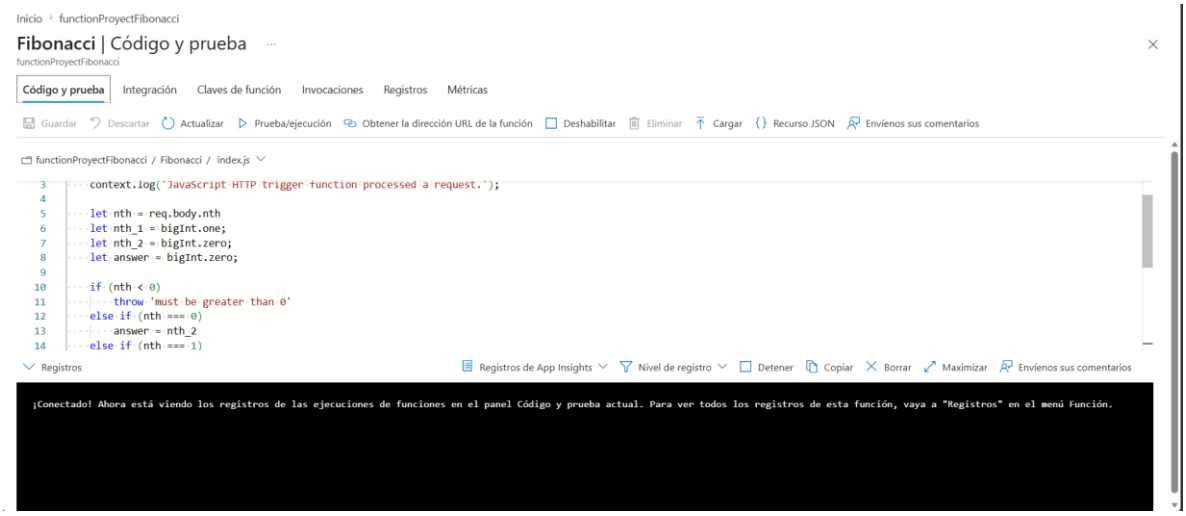
Suscripción	e646e614-bc9c-4092-bc48-895cab752921
Grupo de recursos	scalability_lab_2
Nombre	functionProyectFibonacci
Nombre de host predeterminado único seguro	Habilitado
Pila del entorno en tiempo de ejecución	Node.js 22 LTS
Tamaño de instancia	2048 MB

Crear | < Anterior | Siguiente > | Descargar una plantilla para la automatización

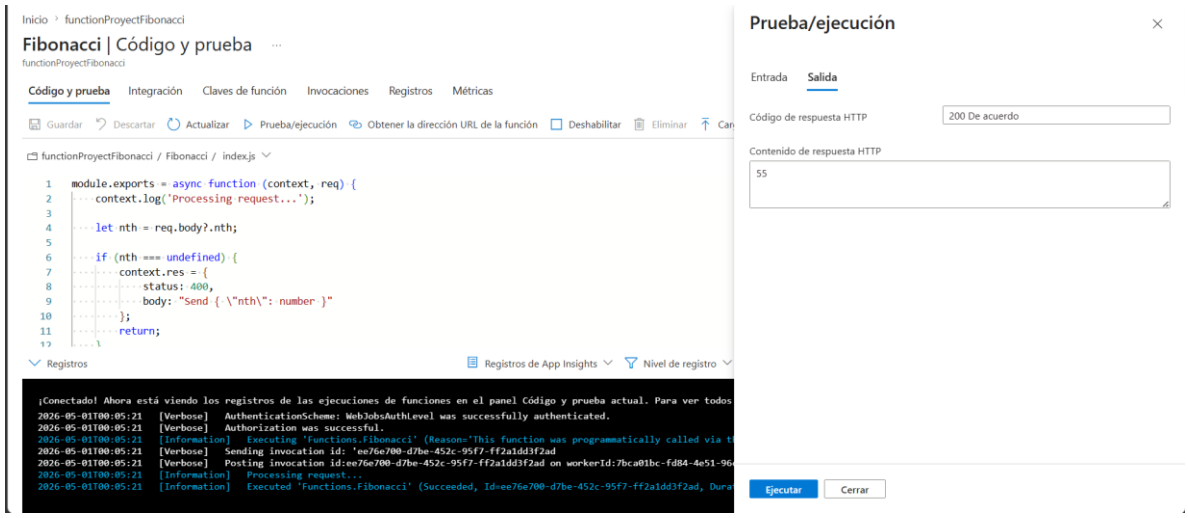
2 y 3) Importación de la aplicación a azure functions de visual studio

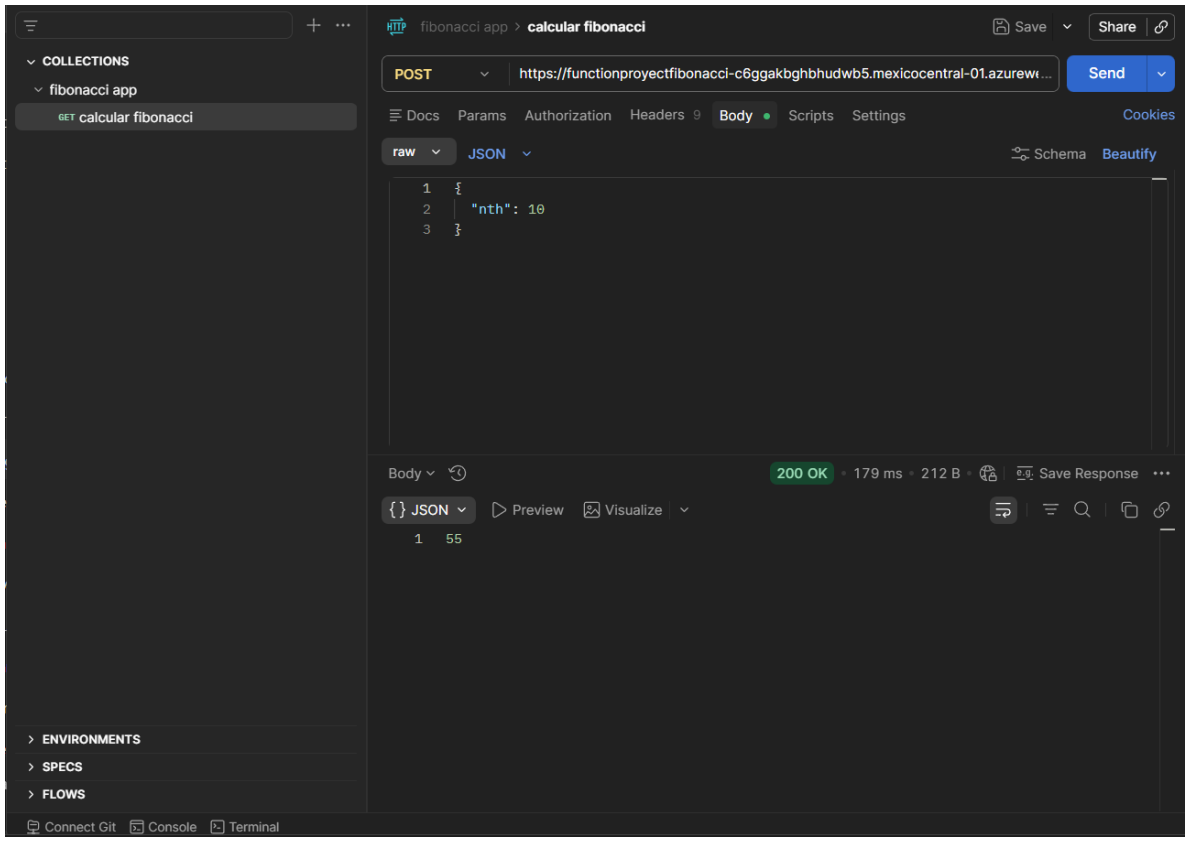


Aplicación importada



4) Test de la aplicacion





5) Uso de Newman para enviar 10 peticiones recurrentes, estos son los resultados

```
C:\WINDOWS\system32\cmd.

Iteration 9/10

→ New Request
  POST https://functionproyectfibonacci-c6ggakbghbhudwb5.mexicocentral-01.azurewebsites.net/api/Fibonacci?code=zJcGm4yJJ
g3Wnuv6AUTohlO-AUUmT4JmWJv3mS83MT25AzFutt8SgQ== [200 OK, 214B, 109ms]

Iteration 10/10

→ New Request
  POST https://functionproyectfibonacci-c6ggakbghbhudwb5.mexicocentral-01.azurewebsites.net/api/Fibonacci?code=zJcGm4yJJ
g3Wnuv6AUTohlO-AUUmT4JmWJv3mS83MT25AzFutt8SgQ== [200 OK, 214B, 111ms]
```

	executed	failed
iterations	10	0
requests	10	0
test-scripts	0	0
prerequest-scripts	0	0
assertions	0	0
total run duration: 2.9s		
total data received: 31B (approx)		

Informe
Se utilizó Postman para definir las solicitudes HTTP y Newman para ejecutarlas desde la línea de comandos. Se configuró un archivo de datos (data.json) con 10 valores distintos para el cálculo de Fibonacci. Posteriormente, se ejecutaron múltiples instancias de Newman en paralelo para simular concurrencia.

Resultados:

- Iteraciones: 10
- Requests: 10
- Fallos: 0
- Tiempo promedio: 214 ms
- Tiempo mínimo: 109 ms
- Tiempo máximo: 1143 ms

Se observó un tiempo alto en la primera petición debido al cold start, mientras que las siguientes solicitudes mantuvieron tiempos estables.

Conclusión:
La función responde correctamente y de forma estable bajo múltiples solicitudes. El sistema muestra un buen desempeño, con tiempos de respuesta consistentes después de la inicialización.

6) Tras implementar un enfoque recursivo de memorizacion se realizaron muchas pruebas con los mismos valores anteriores y los resultados son los siguientes

```
→ New Request
  POST https://functionproyectfibonacci-c6ggakbghbhudwb5.mexicocentral-01.azurewebsites.net/api/FibonacciMemo?code=GGkjI
0-5Awgx4jb-w6ZxRiXHf4LYDm1SqGEsPwmg42K4AzFusiQObg== [200 OK, 214B, 112ms]

Iteration 10/10

→ New Request
  POST https://functionproyectfibonacci-c6ggakbghbhudwb5.mexicocentral-01.azurewebsites.net/api/FibonacciMemo?code=GGkjI
0-5Awgx4jb-w6ZxRiXHf4LYDm1SqGEsPwmg42K4AzFusiQObg== [200 OK, 214B, 115ms]
```

	executed	failed
iterations	10	0
requests	10	0
test-scripts	0	0
prerequest-scripts	0	0
assertions	0	0
total run duration: 2.5s		
total data received: 31B (approx)		
average response time: 169ms [min: 112ms, max: 669ms, s.d.: 166ms]		

Como se puede observar el tiempo de respuesta es casi la mitad de la implementación pasada, por lo que el enfoque recursivo mejoro el rendimiento

Luego tras dejar esperar algunos minutos sin realizar peticiones y volver a hacer peticiones, se puede observar un tiempo diferente de respuesta mas elevado que el anterior, esto debido al cold start, es decir cuando una función en la nube tarda más en responder porque no estaba activa y el sistema tiene que arrancarla desde cero

```
→ New Request
  POST https://functionproyectfibonacci-c6ggakbghbhudwb5.mexicocentral-01.azurewebsites.net/api/FibonacciMemo?code=GGkjI
0-5Awgx4jb-w6ZxRiXHf4LYDm1SqGEsPwmg42K4AzFusiQObg== [200 OK, 214B, 125ms]

Iteration 10/10

→ New Request
  POST https://functionproyectfibonacci-c6ggakbghbhudwb5.mexicocentral-01.azurewebsites.net/api/FibonacciMemo?code=GGkjI
0-5Awgx4jb-w6ZxRiXHf4LYDm1SqGEsPwmg42K4AzFusiQObg== [200 OK, 214B, 116ms]
```

	executed	failed
iterations	10	0
requests	10	0
test-scripts	0	0
prerequest-scripts	0	0
assertions	0	0
total run duration: 2.7s		
total data received: 31B (approx)		
average response time: 206ms [min: 116ms, max: 994ms, s.d.: 262ms]		

Preguntas

1. Azure function es un servicio de azure para ejecutar pequeñas funciones sin necesidad de una gran infraestructura como lo pueden ser peticiones HTTP, cambios en base de datos, mensajes de colas etc
2. Serverless es un modelo de computación donde el desarrollador sube el código sin gestionar servidores, el proveedor se encarga de eso
3. El runtime es el entorno de ejecución donde corre la función elegir el runtime implica el lenguaje que se usara, librerías, versiones y rendimiento
4. Azure lo usa para almacenar metadatos, gestionar triggers, mantener estado de ejecución y logs
5.
   - **Consumption Plan**: Pago por ejecución, escalado automático, Sin infraestructura dedicada
   - **Premium Plan**: Instrancias pre-calentadas, escalado automático mas rápido sin cold start
   - **App Service Plan**: Infraestructura dedicada, comparte recursos con otros apps hay control total
6. Memoization puede fallar por que azure function es stateless, el cold start borra memoria previa toca usar cache externo
7. El consuption plan se cobra por ejecuciones y tiempo de ejecucion